UK BANKING · WHAT THE MODELS ACTUALLY DO

# I built three risk engines for a UK retail bank.

## The interesting part was where each one stops working.

IFRS 9 provisioning under stress, a treasury structural hedge, and graph detection of money mule networks. Built, tested and documented, on synthetic data throughout.

Four things the standard specification of these models gets wrong, and what happens when you fix them.

Nihal Naresh Kumar · MSc Mathematical Finance

August 2026

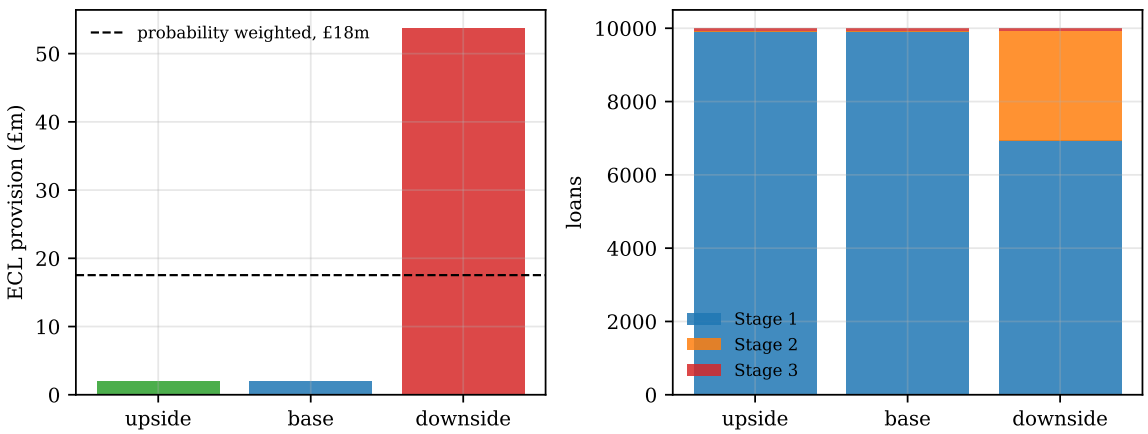
Code, tests and full write-up: link in the comments

# The cliff nobody warns you about

01. IFRS 9 PROVISIONING

A bank holds a twelve-month loss allowance against a performing loan. If credit risk has increased significantly since origination, the loan moves to Stage 2 and the allowance becomes *lifetime*. The provision does not rise smoothly. It jumps.

On a synthetic book of 10,000 mortgages worth £2.80bn:




The severe downside raises the provision from £2.0m to £53.7m, a factor of **26**. Almost none of that is higher loss rates. It is stage migration: 2,991 loans, just under 30% of the book, breach the test and move to a lifetime allowance. Mean provision per loan goes from £228 to £17,224.

**This is why bank provisions move so violently in a downturn. The accounting standard contains a discontinuity, and the macro scenario decides which side of it you are on.**

# A modelling error that hides the whole effect

02. WHAT WENT WRONG FIRST

My first run produced *zero* Stage 2 loans, even in the severe downside. The engine looked fine. It was not.

The cause: applying the same macroeconomic shift to every loan's log-odds of default moves every PD by the same ratio. The staging test compares current PD to origination PD, so if every ratio is identical, the test either fires for the entire book or for none of it. There is no migration, only a switch.

The fix is a loan-level sensitivity that rises with loan-to-value and with the debt service burden, so a highly geared borrower migrates faster than a lightly geared one under the same scenario:

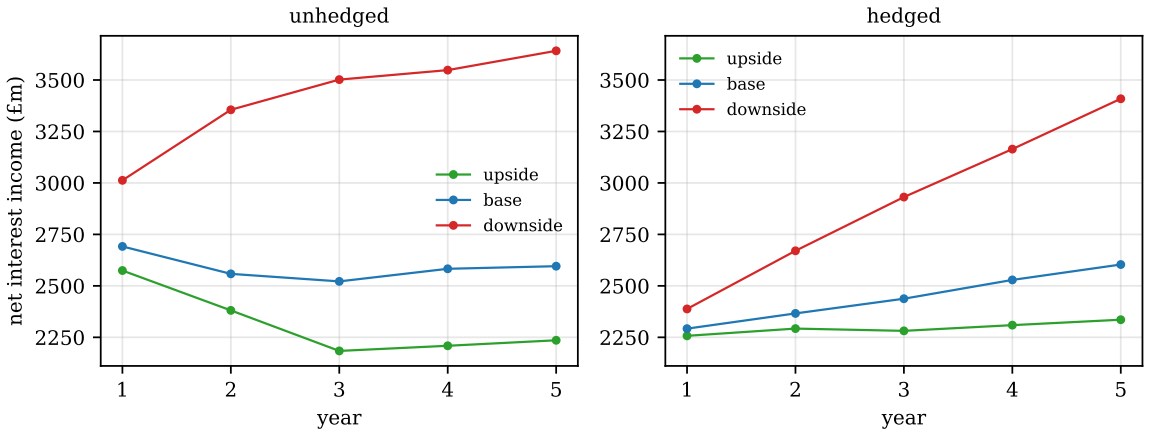
$$\text{logit}(PD_t) = \text{logit}(PD_0) + s_i\,(\beta_1 \Delta u_t - \beta_2(\text{HPI}_t - 1))$$

**A model that runs, produces plausible totals and is silently wrong is more dangerous than one that crashes. The only reason I caught this was checking the stage distribution rather than the headline number.**

# A hedge is a delay, not a cure

03. Treasury structural hedging

A bank funded by deposits that do not reprice is exposed when rates move. The standard defence is a rolling ladder of receiver swaps: one tranche matures and is replaced each year, so the book receives a moving average of past fixed rates while paying today's floating rate.




Across three macro scenarios, the ladder absorbs **70%** of the spread in net interest income in year 1. By year 5 it absorbs **24%**.

The protection decays because the ladder rolls. Old tranches struck at yesterday's rates mature and are replaced at today's, so over its own tenor the hedge converges towards the unhedged position.

**The structural hedge does not remove rate risk. It buys the bank about five years to reprice its business, which is a completely different claim.**

# Sizing it wrong makes it look useless

04. THE SECOND THING I GOT WRONG

My first ALM run showed the hedge absorbing only 10% of the swing, which would make the entire treasury practice look pointless. Two errors, both in the setup rather than the mathematics.

**The notional was sized on the whole deposit base.** Only the *rate-insensitive* portion creates the structural exposure. Deposits that reprice with the market are already hedged by construction, so including them over-states the base and under-states the ratio.

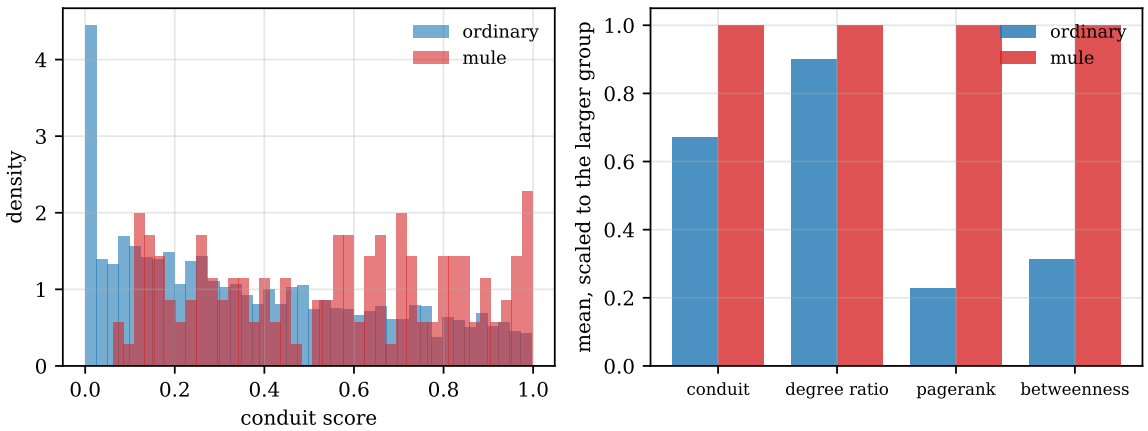
**The ladder started empty.** A ladder built from nothing holds one tranche in year 1 and does almost nothing. Real treasury books are already mature, with tranches struck over the preceding five years. Starting from zero flatters the unhedged comparison.

**Both errors made the hedge look ineffective. Neither was in the formula. Setup assumptions decided the answer, which is usually where the risk in a model sits.**

# When a good score is a warning sign

05. GRAPH FRAUD DETECTION

Money mule rings move funds through chains of accounts, so the signal is topological rather than transactional. I built a payment graph of 3,000 accounts and 7,831 payments with 25 planted rings, computed degree ratios, PageRank, betweenness and a conduit score, and trained a classifier.




It scored an ROC AUC of 0.997 and recovered every mule in the test set inside the top 100 alerts.

That number is not a success. It is a warning. The rings were planted with a distinctive topology, so what the score measures is whether the features recover a structure I put there myself. A real mule network is adversarial, sparser and deliberately shaped to look ordinary.

**The honest claim is that the pipeline is correct, not that it is accurate. Any fraud model reporting 0.99 on synthetic data is reporting the quality of its own simulator.**

# What is actually in the repository

06. ENGINEERING

**Three modules.** IFRS 9 with macro-adjusted PD, a collateral haircut LGD, amortising EAD and SICR staging. ALM with behavioural deposit decay and the swap ladder. A graph feature pipeline with a gradient boosted classifier.

**34 unit tests.** Not coverage theatre. They check that PD stays inside $(0, 1)$ under extreme inputs, that ECL is never negative, that lifetime ECL is never below twelve-month ECL, that the receiver ladder gains when rates fall and loses when they rise, that a zero hedge ratio is a no-op, and that a flat rate path leaves the ladder exactly neutral.

**Reproducible.** Every seed is fixed. One command regenerates every number on these slides.

**Documented honestly.** The README carries a section listing the four places I departed from the original specification and why, and a limitations section stating what the project does not model.

**I would rather be asked about the limitations section than the results table. It is the part that took judgement.**

# Three things worth pushing back on

07. And what I would say back

### 1. “It is all synthetic. This proves nothing.”

Correct, and the README says so at the top. What synthetic data can establish is that the mechanics are implemented properly and that the model behaves as theory says it should. What it cannot establish is any number’s accuracy. Conflating those two is the failure mode I was trying to avoid.

### 2. “Three unrelated domains in one repository is scope creep.”

Largely fair. Credit risk, treasury and financial crime sit in different parts of a bank and rarely in one job description. My defence is that they share a balance sheet and a data generator, and that the common theme is model risk rather than the domains. If I were cutting one, it would be the fraud engine.

### 3. “Your macro scenarios are imposed, not generated.”

True, and it is the weakest part. The paths are deterministic ramps with noise, so the tail is assumed rather than emergent. A proper treatment would use a stochastic scenario generator, and the stage migration figures would then come with a distribution rather than a point.

**If any of this is wrong, I would rather hear it in the comments than find out in an interview.**

# The lesson I did not expect

08. Closing

I set out to implement three models. What I spent most of the time on was catching the cases where they ran perfectly well and gave the wrong answer.

Zero Stage 2 loans in a severe recession. A structural hedge that appeared to do nothing. A fraud classifier scoring 0.997. All three produced clean output. All three were artefacts of an assumption made somewhere outside the formula, in how the inputs were shaped or how the exposure was sized.

None of them would have been caught by a unit test on the mathematics, because the mathematics was right in every case.

## The question I keep coming back to:

In model validation, how much of the real work is checking the equations, and how much is interrogating the assumptions that surround them? My experience here says the second, by some distance. I would be interested to hear from people who do this for a living.

Repository with the code, the tests and the full write-up in the comments.
Corrections are welcome, particularly on the IFRS 9 staging treatment.

# References and stack

09. Sources

**Standards and supervisory material**

IFRS 9 *Financial Instruments*, IASB (2014). The classification, the SICR test and the twelve-month against lifetime allowance distinction.

Bank of England, *Stress testing the UK banking system*. The structure of base, upside and severe downside scenarios.

Basel Committee, *Interest rate risk in the banking book* (2016). The framework behind the net interest income sensitivity measured here.

**Method**

Chawla, Forest and Aguais, Some options for evaluating significant deterioration under IFRS 9, *Journal of Risk Management in Financial Institutions* (2016). Approaches to the SICR trigger.

Memmel, Bank interest rate risk and the structural hedge. On sizing a ladder against non-maturing deposits.

Weber, Chen and others on graph-based anti-money-laundering. Topological features for detecting layering in payment networks.

**Stack**

Python 3.10 to 3.12, NumPy, pandas, SciPy, scikit-learn, NetworkX, Matplotlib, Streamlit, pytest, Docker, GitHub Actions.

All data is synthetic and generated by the repository itself. Nothing here is calibrated to a real bank, a real portfolio or a real payment network, and no figure should be read as an estimate.